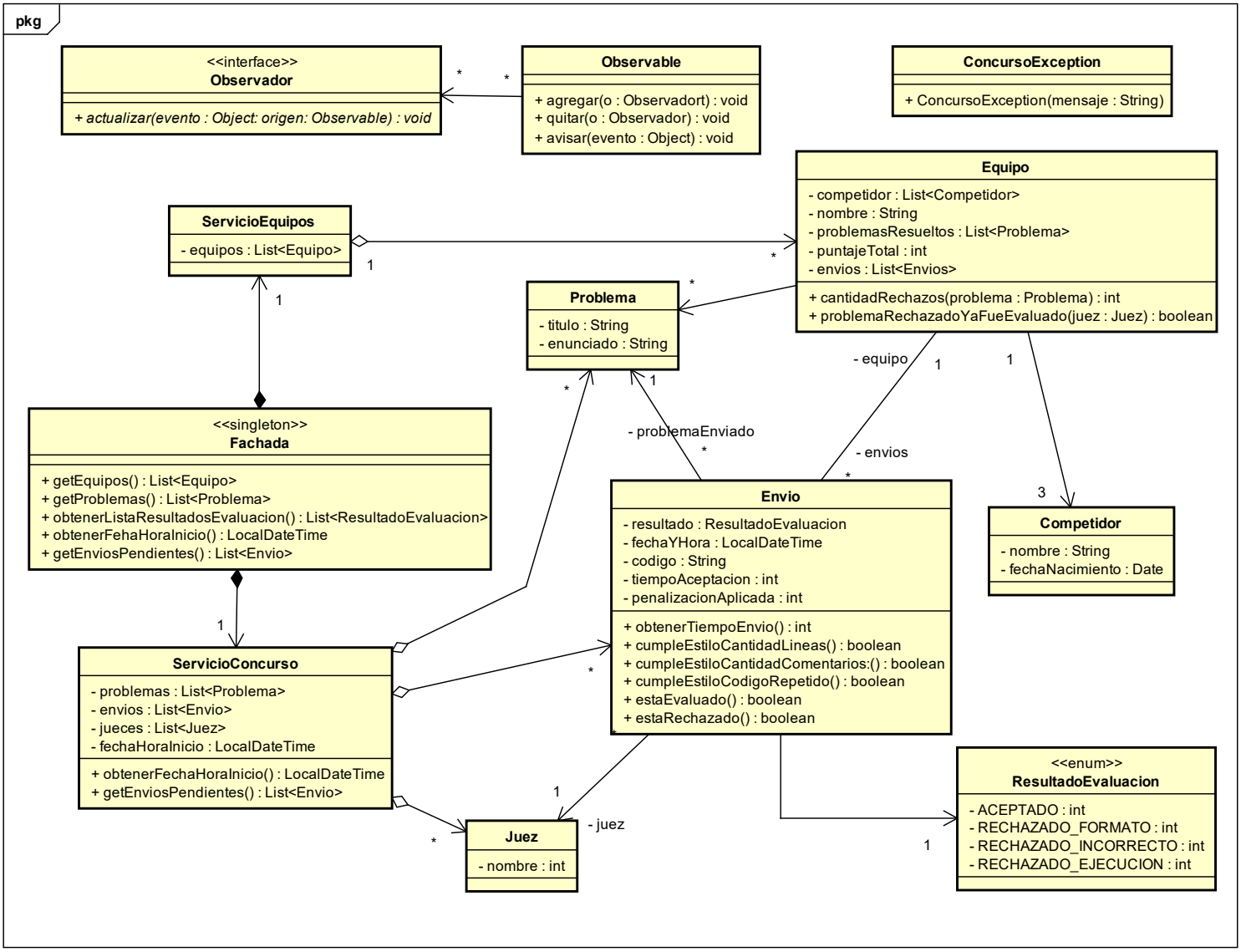
UNIVERSIDAD ORT
Uruguay

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel 902 15 05 Fax 908 13 70
www.ort.edu.uy

EVALUACION	**SOLUCION PARCIAL**	**GRUPO**	N4A-N4C-N4D	**FECHA**	27/11/2025
MATERIA	DISEÑO DE APLICACIONES / DISEÑO Y DESARROLLO DE APLICACIONES				
CARRERA	ANALISTA EN TECNOLOGIAS DE INFORMACIÓN / ANALISTA PROGRAMADOR				
CONDICIONES	**- Puntos: 40** **- Duración: 2 HORAS** **- Sin Material** **-Debe entregar esta letra junto con la prueba**				

Dado el siguiente diagrama de diseño que corresponde a un sistema de gestión de un concurso de programación en JAVA.



Nota: Los métodos que figuran en este diagrama de diseño se asumen ya implementados.

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel 902 15 05 Fax 908 13 70
www.ort.edu.uy

Los equipos, formados por tres competidores, participan en el concurso resolviendo problemas definidos.

Cuando un equipo envía una solución para evaluar, un juez registra el resultado de la evaluación**.**

El resultado de la evaluación puede ser:

- **Aceptado**: El problema fue resuelto correctamente
- **Rechazado por formato**: La salida no cumple con el formato esperado
- **Rechazado por respuesta incorrecta**: La solución produce resultados incorrectos
- **Rechazado por ejecución**: Error de tiempo de ejecución o compilación

El sistema de puntuación funciona de la siguiente manera:
Cuando un problema es aceptado:

- Se suma al equipo el tiempo transcurrido desde el inicio del concurso hasta la aceptación (en minutos)
- Se incrementa el contador de problemas resueltos del equipo
- Se suman las penalizaciones acumuladas por intentos fallidos previos de ese problema que correspondan. La forma de calcular la penalidad depende del problema enviado.
- La penalidad no se suma al puntaje hasta que el problema sea aceptado.

Cuando un problema es rechazado:

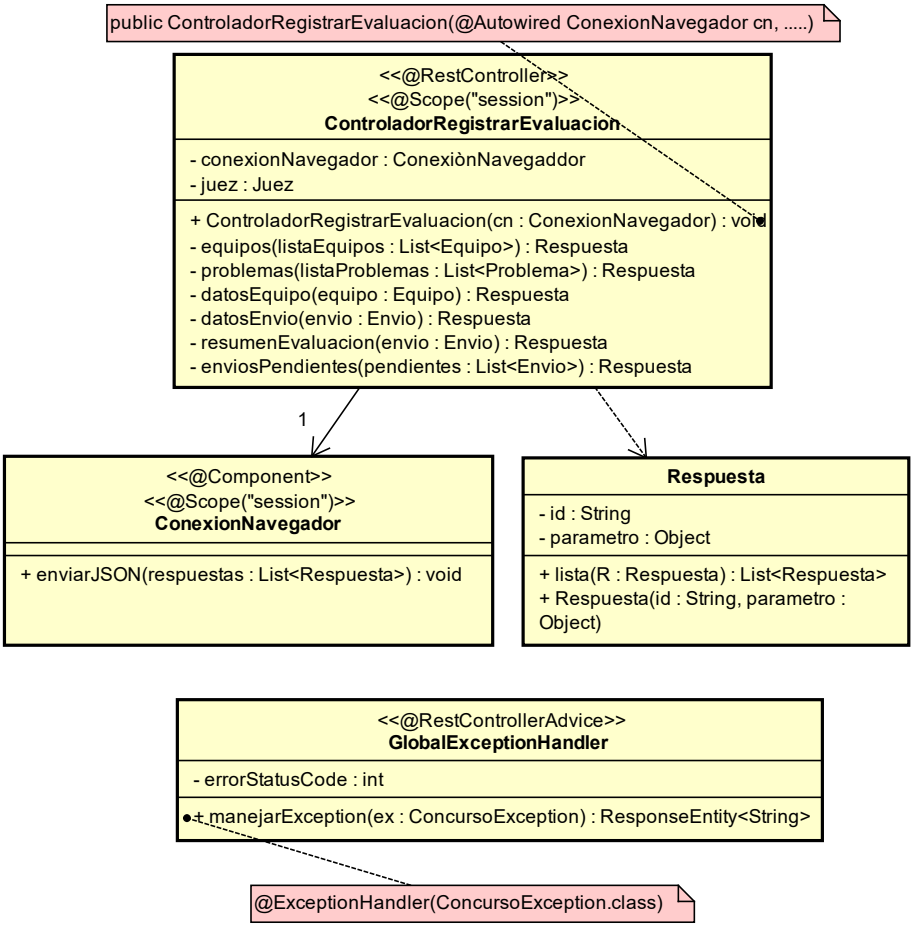
- Se registra en el envío el motivo de rechazo.
- El equipo puede seguir intentando resolver el mismo problema

Por el momento existen 3 formas de penalización definidas para los envíos que son rechazados (más adelante se podrán incluir nuevas formas de penalización).

- **Estándar**: Cada rechazo suma 20 minutos de penalidad.
- **Exigente**: Cada rechazo suma 20 minutos, pero si el equipo tiene más de 3 rechazos en el mismo problema, la penalidad se incrementa a 30 minutos por cada rechazo adicional.
- **Progresiva**: La penalidad aumenta con cada rechazo del mismo problema: primer rechazo 10 minutos, segundo rechazo 20 minutos, tercer rechazo 30 minutos, y así sucesivamente (incremento de 10 minutos por cada rechazo).

La forma de penalización está asociada a cada problema y puede ser modificada durante el transcurso del concurso a criterio de los jueces solo si el problema aun no fue enviado por ninguno de los equipos participantes (para generar más adrenalina 😊).

Se cuenta con el siguiente controlador con los métodos que reciben objetos del dominio y que retornan objetos Respuesta ya implementados, en donde se generan los objetos DTO con la información necesaria para cada respuesta.




```
@RestController
@RequestMapping("/registrarEvaluacion")
@Scope("session")
public class ControladorRegistrarEvaluacion {
   private final ConexionNavegador conexionNavegador;
   private Juez juez;
   ……
   // Incluir aquí los atributos y endpoints que considere
  // necesarios para la solución

}
```

CU: Registrar Evaluación de Problema

Precondiciones: El CU recibe el juez como parámetro en el constructor.

Curso normal:

Juez	**Sistema**
	1. Muestra una lista con los envíos pendientes de evaluación.
2. Selecciona un envío.	3. Muestra el nombre del equipo, la cantidad de problemas resueltos, el puntaje acumulado y la cantidad de envíos pendientes.
	4. Muestra el código y título del problema, y la cantidad de intentos fallidos del equipo.
5. Selecciona el resultado de la evaluación (Aceptado, Rechazado por formato, Rechazado por respuesta incorrecta, Rechazado por ejecución).	
6. Confirma que desea registrar la evaluación.	7. Muestra un resumen con el tiempo transcurrido, el resultado de la evaluación y, si aplica, el puntaje que se sumará al equipo.
	8. Actualiza el envío, actualiza el puntaje del equipo (si fue aceptado) y registra el juez que realizó la evaluación.

Cursos Alternativos:

Si un problema enviado ya fue rechazado previamente (sin importar el motivo), no puede ser evaluado por un juez que ya ha participado en su evaluación. En caso de que se intente registrar una evaluación que no cumpla esa condición no se registra y se despliega el mensaje "El juez que realizó la evaluación no está habilitado para evaluar este envío"

Independientemente del resultado de la evaluación, si al momento de registrar la evaluación se detecta que el código del problema no sigue las reglas de estilo del concurso, se despliega un mensaje "Advertir al equipo que debe mejorar el estilo de codificación". Para que el código siga las reglas de estilo debe:

1) Tener entre 10 y 200 líneas. [envio.cumpleEstiloCantidadLineas()]
2) Contener al menos 10 comentarios [envio.cumpleEstiloCantidadComentarios()]
3) El nombre de la clase que contiene el main debe llamarse Principal y no tener código repetido. [envio.cumpleEstiloCodigoRepetido()]

Nota: Observar que la clase envío tiene 3 métodos ya implementados que evalúan las reglas de estilo indicadas.

IMPORTANTE: **Este caso de uso puede ejecutarse simultáneamente por otros jueces** por lo que debe asegurarse que **la lista de envíos pendientes de evaluación se actualice automáticamente para todas las vistas de jueces** que estén evaluando problemas del equipo sobre el cual se confirmó una evaluación.

Se pide:

a) Implemente en Java el caso de uso descrito, **exceptuando la vista,** sí debe implementar el controlador necesario. No es necesario implementar constructores, ni métodos accesores y modificadores ***salvo que sean necesarios para la comprensión de la solución.***
No es necesario implementar las clases Fachada , ConcursoException y GlobalHandlerException. Tampoco Observable ni la interface Observador. Tampoco es necesario implementar otras clases que no utilice en su solución o que solo contengan métodos get y set. No debe implementar los DTO que necesite para su solución y tampoco los métodos privados que generan los objetos Respuesta indicados en el diagrama UML en la clase ControladorRegistrarEvaluación.

Si bien el diseño debe incluir las 3 formas de penalización indicadas solo **se pide implementar el código de al menos dos** de las tres (a elección).

Requerimientos de diseño:

- Utilizar **estrictamente** el diagrama de clases dado en la letra como **base** para su solución.
- Solo debe agregar lo estrictamente necesario para la solución del caso de uso.
- Aplicar los siguientes conceptos, principios y patrones:
  - Utilizar el principio de Experto para asignar las responsabilidades a las clases. (7 pts.)
  - División lógica. (5 pts.)
  - Aplicar MVC (8 pts.)
  - Manejo de eventos (7 pts.)
  - Manejo de Excepciones (3 pts.)
  - Diseño e implementación aplicando polimorfismo (7 pts.)

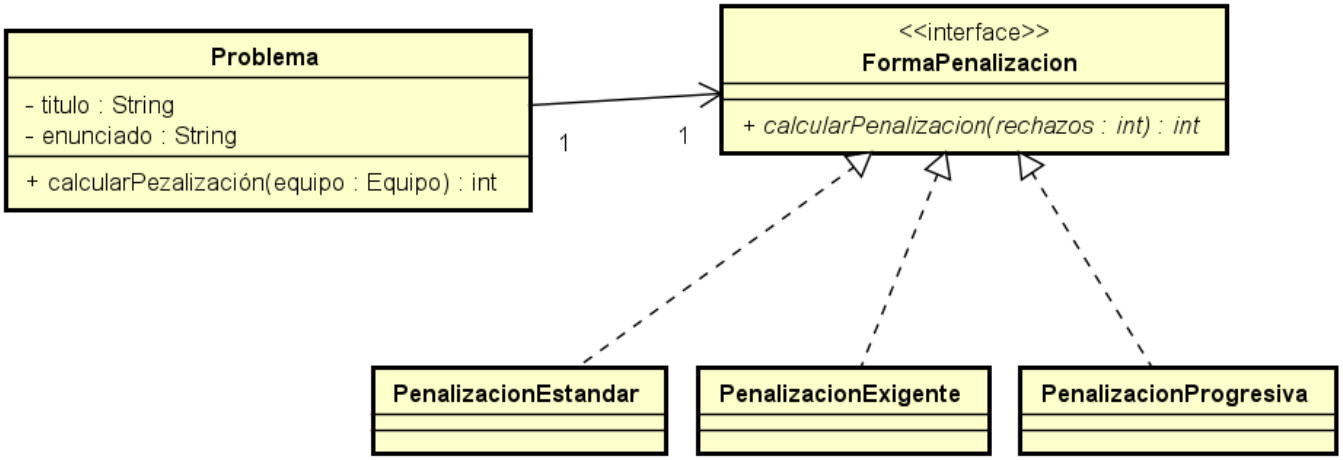
b) Dibuje **las clases que haya agregado en su solución en notación UML** incluyendo las firmas de los métodos implementados. Si las clases están asociadas a alguna de las clases indicadas en la letra también debe incluirlas en el dibujo **(3 puntos)**.

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel 902 15 05 Fax 908 13 70
www.ort.edu.uy

POSIBLE SOLUCION Parte A) :

```
public class ControladorRegistrarEvaluacion implements Observer {
   private Juez juez;
   private Equipo equipo;
   private Envio envio;
   private List<Envio> listaEnvios;
   @PostMapping("/vistaConectada") // Inicializa la vista mostrando los equipos (el juez ya vino en el constructor)
   public List<Respuesta> inicializarVista() throws ConcursoException {
      listaEnvios = Fachada.getInstancia().getEnviosPendientes();
      return Respuesta.lista(equipos(enviosPendientes(this.listaEnvios));
   }

   @PostMapping("/seleccionarEnvio")   // 3) El juez selecciona un envío
   public List<Respuesta> seleccionarEnvio(@RequestParam int posEnvio) throws ConcursoException {
      Envio unEnvio = listaEnvios.get(posEnvio);
      if (envio != null) {
         envio.quitarObservador(this);
      }
      envio = unEnvio;
      envio.agregarObservador(this);
      equipo = unEnvio.getEquipo();
      // La vista mostrará código, título y cantidad de intentos fallidos a partir del DTO
      return Respuesta.lista(datosEquipo(equipo), datosEnvio(envio));
   }

   @PostMapping("/registrarEvaluacion")  // 6–8) Registrar la evaluación del envío seleccionado
   public List<Respuesta> registrarEvaluacion(@RequestParam int posEnvio,
                                              @RequestParam String resultado)  throws ConcursoException {
      Envio envio = pendientes.get(posEnvio);
      ResultadoEvaluacion res = ResultadoEvaluacion.valueOf(resultado);
      envio.registrarEvaluacion(res,juez);

      return Respuesta.lista(resumenEvaluacion(envio));
   }

   // Observador: actualiza automáticamente la lista de envíos pendientes para el equipo
   @Override
   public void actualizar(Object evento, Observable origen) {
      if (evento.equals(Equipo.Eventos.ENVIO_EVALUADO)) {
         conexionNavegador.enviarJSON(
               Respuesta.lista(
                     enviosPendientes(equipo.getEnviosPendientes()),
                     datosEquipo(equipo)));
      }
   }
}
```

```
public class Envio extends Observable {
   public enum Eventos {
      ENVIO_EVALUADO
   }
   public void marcarAceptado(Juez juez, int tiempo, int penalizacion) throws ConcursoException {
      this.resultado = ResultadoEvaluacion.ACEPTADO;
      this.tiempoAceptacion = tiempo;
      this.penalizacionAplicada = penalizacion;
      equipo.confirmarEnvioAceptado(this  , juez, this.tiempoAceptacion + this.penalizacionAplicada);
   }
   public void marcarRechazado(Juez juez, ResultadoEvaluacion resultado) throws ConcursoException {
      this.resultado = resultado;
      equipo.confirmarEnvioRechazado(this, juez, resultado);
   }
   public void registrarEvaluacion(ResultadoEvaluacion res, Juez juezEvaluador) throws ConcursoException {

      if (res == ResultadoEvaluacion.ACEPTADO) {
         marcarAceptado(juezEvaluador, obtenerTiempoEnvio(), problemaEnviado.calcularPenalizacion(equipo));

      } else {
          if (equipo.problemaRechazadoYaEvaluado(envio.getProblema(), juez))
             throw new ConcursoException ("El juez que realizó la evaluación no está habilitado para evaluar este
                                                envío");
         marcarRechazado(juez, res);
      }
      avisar(Eventos.ENVIO_EVALUADO);
      if (!validaEstiloCodigo()) {
         throw new ConcursoException("El codigo de la solución no cumple con las reglas de estilo");
      }
   }
   private boolean validaEstiloCodigo() {
      return this.cumpleEstiloCantidadLineas() &&
            this.cumpleEstiloCantidadComentarios() &&
            this.cumpleEstiloCodigoRepetido();
   }
}

public class Equipo   {
   public void confirmarEnvioAceptado(Envio envio, Juez juez, int puntaje) throws ConcursoException {
      if (envio.estaEvaluado()) throw new ConcursoException("El envío ya fue evaluado");
      problemasResueltos++;
      puntajeTotal += puntaje;
   }
   public void confirmarEnvioRechazado(Envio envio, Juez juez, ResultadoEvaluacion resultado)
         throws ConcursoException {
      if (envio.estaEvaluado()) {
         throw new ConcursoException("El envío ya fue evaluado");
      }
   }
```

```
    public boolean problemaRechazadoYaEvaluado(Problema p, Juez j){
                for (Envio e: listaEnvios){
                        if (e.getProblema() == p && e.estaRechazado() && e.getJuez()==j) return true;
                }
                return false;
        }
}
public class Problema {
   private FormaPenalizacion formaPenalizacion;
   // Delegación del cálculo de penalización en la estrategia polimórfica
   public int calcularPenalizacion(Equipo equipo) {
      int rechazos = equipo.cantidadRechazos(this);
      return formaPenalizacion.calcularPenalizacion(rechazos);
   }
}
//FORMA penalizacion  (La letra solicita implementar solo una de las tres estrategias)
public interface FormaPenalizacion {
   public int calcularPenalizacion(int rechazos);
}
//PENALIZACION ESTANDAR:  cada rechazo suma 20 minutos
public class PenalizacionEstandar extends FormaPenalizacion {
   @Override
   public int calcularPenalizacion(int rechazos) {
      return rechazos * 20;
   }
}
// PENALIZACION EXIGENTE: 20 minutos hasta 3 rechazos, luego 30 por cada rechazo adicional
public class PenalizacionExigente extends FormaPenalizacion {
   @Override
   public int calcularPenalizacion(int rechazos) {
      int primeros = Math.min(3, rechazos);
      int restantes = Math.max(0, rechazos - 3);
      return primeros * 20 + restantes * 30;
   }
}
//PENALIZACION PROGRESIVA: 10, 20, 30, ... (incremento de 10 por cada rechazo)
public class PenalizacionProgresiva extends FormaPenalizacion {
   @Override
   public int calcularPenalizacion(int rechazos) {
      return 10 * rechazos * (rechazos + 1) / 2;
   }
}
```

POSIBLE SOLUCION Parte B) :